Data flow in graphs

Meenakshi D'Souza

International Institute of Information Technology
Bangalore.

NPTEL

Graph coverage criteria: Overview

- We look at graphs as structures and look at the following two coverage criteria over graphs.
  - Structural coverage criteria.
  - Data flow coverage criteria.
- Later, we consider software artifacts (code, design elements and requirements) modelled as graphs and see how these coverage criteria apply to them.
- Focus of this lecture: Model data flow in graphs.

NPTEL

Graphs with data

- Graph models of programs can be tested adequately by including values of variables (data values) as a part of the model.
- Data values are created at some point in the program and used later. They can be created and used several times.
- We deal with definition and use of data values.
- We will define coverage criteria that track a definition(s) of a variable with respect to its use(s).

NPTEL

Definition and use of values

- A definition (def) is a location where a value of a variable is stored into memory.
  - It could be through an input, an assignment statement etc.
- A use is a location where a value of a variable is accessed.
  - It could be assigned to another variable, be a part of an if, while or other conditions etc.
- Values are *carried* from their defs to uses. We call these du-pairs or def-use pairs.
- A du-pair is a pair of locations (l_i, l_j) such that a variable v is defined at l_i and used at l_j.

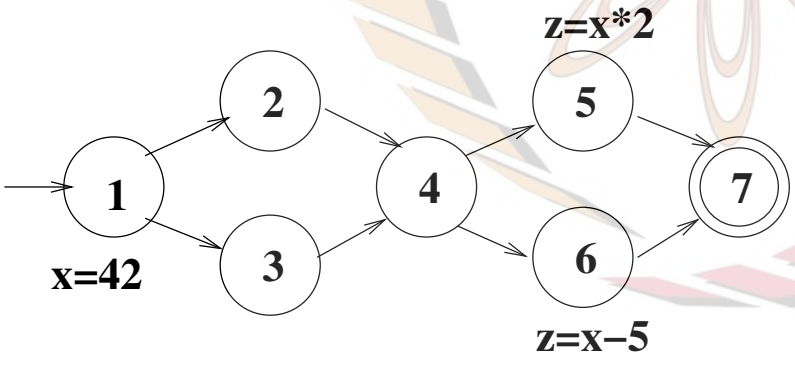
NPTEL

Data in graphs

- Let V be the set of variables that are associated with the program artifact being modelled as a graph.
- The subset of V that each node n (edge e) defines is called $def(n)$ ($def(e)$).
  - Typically, graphs from programs don't have defs on edges.
  - Designs modeled as finite state machines have defs as side effects or actions on edges.
- The subset of V that each node n (edge e) uses is called as $use(n)$ ($use(e)$).

NPTEL

Data in graphs: Example



- Defs:
  - $def(1) = \{x\}$,
  - $def(5) = def(6) = \{z\}$.
- Uses:
  - $use(5) = \{x\}$,
  - $use(6) = \{x\}$.

NPTEL

Example: Pattern Matching

```
public class PatternIndex
{  public static void main (String[] argv)
   {       if (argv.length != 2)
           { System.out.println
             ("java PatternIndex Subject Pattern");
             return; }
           String subject = argv[0];
           String pattern = argv[1];
           int n = 0;
           if ((n = patternIndex(subject, pattern))==-1)
             System.out.println
             ("Pattern is not a substring of the subject");
           else
             System.out.println
             ("Pattern string begins at character " + n);
   }
```

Pattern Matching Example contd.

```
/**
* Find index of pattern in subject string
* @return index (zero-based) of 1st occurrence of pattern
in subject; -1 if not found
* @throws NullPointerException if subject or pattern is
null
*/

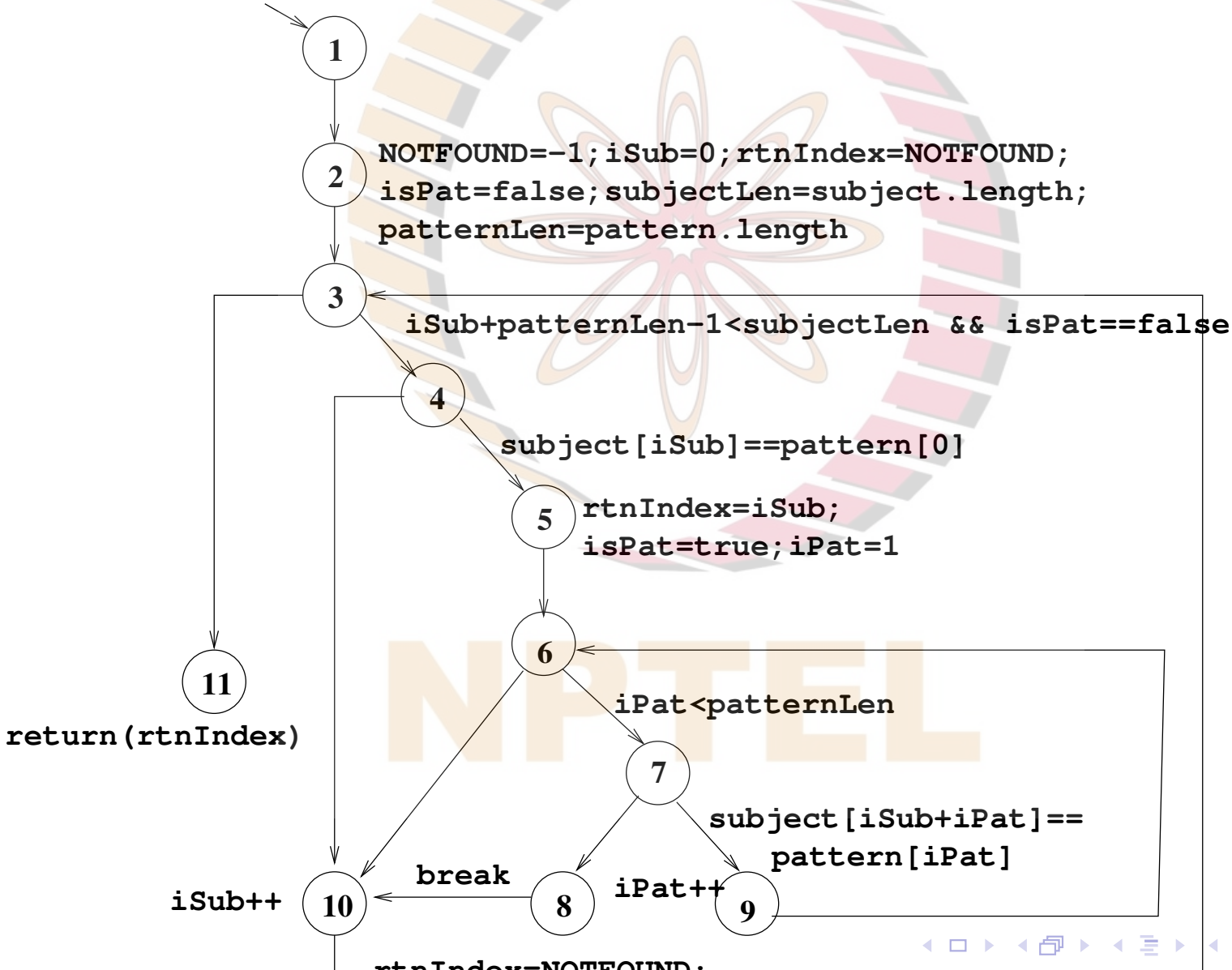
public static int patternIndex(String subject,String pattern)
{
   final int NOTFOUND = -1;
   int iSub = 0, rtnIndex = NOTFOUND;
   boolean isPat = false;
   int subjectLen = subject.length();
   int patternLen = pattern.length();
```

Pattern Matching Example contd.

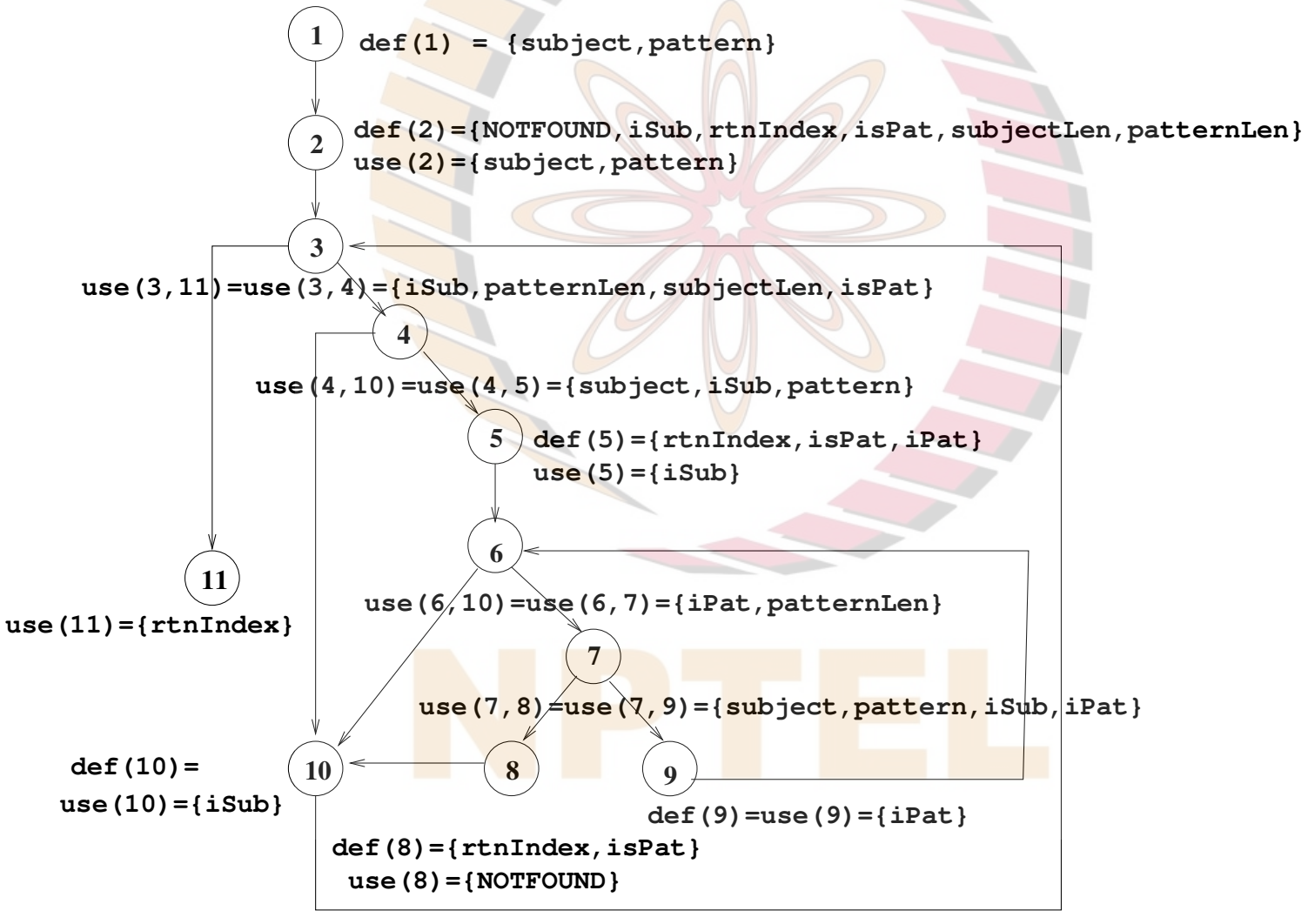
```
while (isPat == false && iSub + patternLen - 1 < subjectLen)
{  if (subject.charAt(iSub) == pattern.charAt(0))
   {  rtnIndex = iSub; // Starting at zero
      isPat = true;
      for (int iPat = 1; iPat < patternLen; iPat ++)
          { if(subject.charAt(iSub+iPat)!=pattern.charAt(iPat))
            { rtnIndex = NOTFOUND;
            isPat = false;
            break; // out of for loop }
   } }
   iSub ++; }
   return (rtnIndex); }
```

NPTEL

Pattern Matching CFG



Pattern Matching: Defs and uses



Pattern Matching: Defs and uses

- The parameters `subject` and `pattern` are forwarded parameters and hence considered to be explicitly defined at node 1.
- Decision statements like `if(subject[iSub]==pattern[0])` result in uses of the variables `subject, iSub, pattern` at the edges (4,5) and (4,10). This is similar for other decision statements too.
- Node 9 *both* defines and uses the variable `iPat` due to the statement `iPat++` which is equivalent to `iPat=iPat+1`.

Data defs and uses: Other definitions

In testing literature, there are two notions of uses available.

- If v is used in a computational or output statement, the use is referred to as a computation use (or c-use), and the pair is denoted as $dcu(l_i, l_j, v)$, where v is defined at l_i and used at l_j.
- If v is used in a conditional statement, its use is called as a predicate use (or p-use).
- For conditional use, two def-use pairs appear:
  - $dpu(l_i, (l_j, l_t), v)$ and $dpu(l_i, (l_j, l_f), v)$, where v is defined at l_i, used at l_j, but has two opposite flow directions (l_j, l_t) and (l_j, l_f).
  - The former denotes the true edge of the conditional statement in which v is used; the latter the false edge.

Data defs and uses

- A def of a variable may or may not reach a particular use.
  - A def of a variable v at location l_i will not reach use of v at location l_j if there is no path from l_i to l_j.
  - The value of v could be changed by another def before it reaches an use.
- A path from l_i to l_j is def-clear with respect to variable v if v is not given another value on any of the nodes or edges in the path.
- If there is a def-clear path from l_i to l_j with respect to v, the def of v at l_i reaches the use at l_j.
- Note: All the path definitions above are parameterized with respect to a variable v.

NPTEL

Def-use paths

- A du-path with respect to a variable v is a simple path that is def-clear from a def of v to a use of v.
  - du-paths are parameterized by v.
  - They need to be simple paths.
  - There may be intervening uses on the path.
- $du(n_i, n_j, v)$: The set of du-paths from n_i to n_j for variable v.
- $du(n_i, v)$: The set of du-paths that start at n_i for variable v.

NPTEL

du-paths: Pattern Matching Example

- There is a du-path for variable `subject` from node 1 to node 2.
  - `subject` is used at node 2 with a reference to its length attribute.
- Similarly, there is a du-path for variable `subject` from node 1 to each of the edges (4,5), (4,10), (7,8) and (7,9).
- Due to the interpretation of `iPat++` as `iPat=iPat+1`, the use occurs before the def. So, a def-clear path goes from node 5 to node 9 for `iPat`.

NPTEL

Moving on... next lecture

We will look at coverage criteria for data flow testing in the next module.

NPTEL

Credits

Part of the material used in these slides are derived from the presentations of the book Introduction to Software Testing, by Paul Ammann and Jeff Offutt.

NPTEL